

ASSESSMENT: TOOL 3 -DATA INTEGRATION TEST

Course Code: DSA0504

**Course Title: Query Processing for
Data Science**

CO Assessed: CO3

**Assessment: Tool 3 -DATA
INTEGRATION TEST**

Weightage: 30

Total Marks: 100 Marks

CO3 Statement: Use Python and Pandas libraries to load, inspect, clean, standardize, integrate, and validate multiple CSV datasets using appropriate keys and join strategies for reliable data analysis. (BL6)

Student Name: Jeya Harshini R

Reg. No.:192424051

Section / Batch:2024

Date of Submission:10/8/26

Faculty In-charge: Dr. R.Mahesh Muthulakshmi

Project Mode: Individual Submission

Experiment Title

Integration of College Student Datasets Using Python and Pandas

<https://colab.research.google.com/drive/1Zua7gtAaf23iq2UfUwaLGn9SjPoSflzt?usp=sharing>

1. AIM

To inspect, clean, standardize, and integrate multiple college student datasets using Python and Pandas by identifying the most reliable common key and resolving structural inconsistencies.

2. OBJECTIVE

- Load multiple CSV datasets using Pandas.
 - Inspect the structure and characteristics of each dataset.
 - Identify missing values and duplicate records.
 - Detect duplicate and inconsistent identifiers.
 - Identify differences in column names and data types.
 - Detect inconsistent categorical values and unmatched records.
 - Standardize the common identifier before integration.
 - Determine the most reliable key for integration.
 - Integrate the datasets using an appropriate merge strategy.
 - Validate the integrated dataset after each major merge.
 - Export the final integrated dataset as Final_Integrated_Dataset.csv.
-

3. SOFTWARE REQUIREMENTS

- Python 3.x
 - Google Colab
 - Pandas
 - NumPy
-

4. INPUT DATASETS

Four CSV datasets were used:

1. Student_Details.csv
2. Academic_Performance.csv
3. Technical_Skills.csv
4. Placement_Details.csv

The datasets contain student demographic details, academic performance, technical skills, and placement information.

5. PROCEDURE

Step 1: Import Required Libraries

The Pandas and NumPy libraries were imported for data loading, inspection, cleaning, and integration.

Code used:

```
[1] ✓ % import pandas as pd
import numpy as np
import os
```

Step 2: Load All CSV Datasets

All four datasets were loaded using `pd.read_csv()` without modifying the original CSV files.

Code used:

```
[2] ✓ 10s from google.colab import files

uploaded = files.upload()

Q1_College...Dataset.zip
Q1_College_Data_Integration_Dataset.zip(application/x-zip-compressed) - 25044 bytes, last modified: 8/10/2026 - 100% done
Saving Q1_College_Data_Integration_Dataset.zip to Q1_College_Data_Integration_Dataset.zip

[3] ✓ 0s import zipfile

zip_file = "Q1_College_Data_Integration_Dataset.zip"

with zipfile.ZipFile(zip_file, "r") as zip_ref:
    zip_ref.extractall("college_data")

print("Files extracted successfully.")

Files extracted successfully.

[4] ✓ 0s os.listdir("college_data")

['Placement_Details.csv',
 'Technical_Skills.csv',
 'Academic_Performance.csv',
 'Student_Details.csv']

[5] ✓ 0s student_df = pd.read_csv(
    "college_data/Student_Details.csv"
)

academic_df = pd.read_csv(
    "college_data/Academic_Performance.csv"
)

skills_df = pd.read_csv(
    "college_data/Technical_Skills.csv"
)

placement_df = pd.read_csv(
    "college_data/Placement_Details.csv"
)

print("All datasets loaded successfully.")

All datasets loaded successfully.
```

Step 3: Inspect Dataset Structure

The shape, column names, data types, and sample records of every dataset were examined before integration.

Code used:

```
[6]
✓ Os
datasets = {
    "Student Details": student_df,
    "Academic Performance": academic_df,
    "Technical Skills": skills_df,
    "Placement Details": placement_df
}

for name, df in datasets.items():

    print("\n" + "="*70)
    print(name)
    print("="*70)

    print("Rows:", df.shape[0])
    print("Columns:", df.shape[1])

    print("\nColumn Names:")
    print(df.columns.tolist())

    print("\nData Types:")
    print(df.dtypes)

    print("\nFirst 5 Records:")
    display(df.head())
```

Observation:

The four datasets have different structures and use different names for the student identifier. Student_ID, ID, student_id, and StudentID are used in different datasets.

```
=====
Rows: 970
Columns: 5

Column Names:
['student_id', 'Programming_Language', 'SQL_Level', 'Communication', 'Certification']

Data Types:
student_id          float64
Programming_Language  object
SQL_Level           object
Communication        object
Certification        object
dtype: object

First 5 Records:
```

	student_id	Programming_Language	SQL_Level	Communication	Certification
0	NaN	Python	Advanced	Excellent	Udemy
1	NaN	Python	Intermediate	Average	NPTEL
2	NaN	Python	Beginner	Excellent	IBM
3	NaN	Python	Beginner	Good	NaN
4	NaN	Java	advanced	good	NaN

```
=====
Placement Details
=====
***
Rows: 1020
Columns: 5

Column Names:
['StudentID', 'Company', 'Package_LPA', 'Placement_Status', 'Placement_Year']

Data Types:
StudentID      float64
Company        object
Package_LPA     float64
Placement_Status object
Placement_Year  int64
dtype: object

First 5 Records:

```

	StudentID	Company	Package_LPA	Placement_Status	Placement_Year
0	NaN	TCS	3.4	Interview	2026
1	NaN	Deloitte	9.9	Not Placed	2026
2	NaN	Cognizant	4.5	Placed	2026
3	NaN	Deloitte	8.7	Placed	2026
4	NaN	Cognizant	3.8	Not Placed	2026

Step 4: Check Missing Values

Missing values were examined in every dataset.

Code used:

for name, df in datasets.items():

```
[7] ✓ 0s for name, df in datasets.items():

    print("\n" + "="*50)
    print(name)
    print("="*50)

    print(df.isnull().sum())
```

Observation:

```

***
=====
Student Details
=====
Student_ID    0
Name          0
Gender        0
Department    0
Year          0
dtype: int64

=====
Academic Performance
=====
ID            5
CGPA          0
Attendance    0
Backlogs      0
Semester      0
dtype: int64

=====
Technical Skills
=====
student_id      5
Programming_Language  0
SQL_Level       0
Communication    0
Certification    192
dtype: int64

=====
Placement Details
=====
StudentID      5
Company        0
Package_LPA    0
Placement_Status 0
Placement_Year 0
dtype: int64

```

Missing values were identified in some datasets, including missing student identifiers. These records were not deleted because the requirement states that records should not be removed without justification.

Step 5: Check Duplicate Records

Complete duplicate rows were identified using Pandas.

Code used:

```

[*]
✓ On  for name, df in datasets.items():

    print(
        name,
        "→ Duplicate complete rows:",
        df.duplicated().sum()
    )

*** Student Details → Duplicate complete rows: 0
Academic Performance → Duplicate complete rows: 0
Technical Skills → Duplicate complete rows: 0
Placement Details → Duplicate complete rows: 0

```

Observation:

Duplicate records were checked separately from duplicate identifiers because two records can have the same student identifier while containing different information.

Step 6: Identify Possible Identifier Columns

Possible identifier columns were identified from each dataset.

Code used:

```
[*]
✓ Os
for name, df in datasets.items():

    print("\n", name)

    possible_ids = [
        col for col in df.columns
        if "id" in col.lower()
    ]

    print("Possible identifier columns:",
          possible_ids)

Student Details
Possible identifier columns: ['Student_ID']

Academic Performance
Possible identifier columns: ['ID']

Technical Skills
Possible identifier columns: ['student_id']

Placement Details
Possible identifier columns: ['StudentID']
```

Observation:

The datasets use different names for the same logical identifier:

Dataset	Identifier
Student Details	Student_ID
Academic Performance	ID
Technical Skills	student_id
Placement Details	StudentID

This demonstrates a structural inconsistency that must be standardized before integration.

Step 7: Check Duplicate and Missing Identifiers

The number of missing, unique, and duplicated identifiers was examined.

Code used:

```
[10] id_columns = {
      "Student Details": "Student_ID",
      "Academic Performance": "ID",
      "Technical Skills": "student_id",
      "Placement Details": "StudentID"
    }

    for name, column in id_columns.items():

        df = datasets[name]

        print("\n" + "-"*60)
        print(name)
        print("ID Column:", column)

        print("Missing IDs:",
              df[column].isna().sum())

        print("Unique IDs:",
              df[column].nunique())

        print("Duplicate IDs:",
              df[column].duplicated().sum())

    """
    Student Details
    ID Column: Student_ID
    Missing IDs: 0
    Unique IDs: 1000
    Duplicate IDs: 0

    Academic Performance
    ID Column: ID
    Missing IDs: 5
    Unique IDs: 980
    Duplicate IDs: 4

    Technical Skills
    ID Column: student_id
    Missing IDs: 5
    Unique IDs: 965
    Duplicate IDs: 4

    Placement Details
    ID Column: StudentID
    Missing IDs: 5
    Unique IDs: 895
    Duplicate IDs: 124
    """
```

Observation:

The analysis identified missing and/or duplicated identifiers in the datasets. These issues were investigated before performing the merge.

Step 8: Compare Identifier Data Types

Code used:

```
[11] ✓ Os
for name, column in id_columns.items():

    print(
        name,
        "→",
        column,
        "→",
        datasets[name][column].dtype
    )

Student Details → Student_ID → int64
Academic Performance → ID → float64
Technical Skills → student_id → float64
Placement Details → StudentID → float64
```

Observation:

The identifier fields were not consistently represented across the datasets. Therefore, their data types were standardized before integration.

Step 9: Standardize Identifier Column Names

All identifier columns were renamed to Student_ID.

Code used:

```
[12] ✓ Os
student_df = student_df.rename(
    columns={"Student_ID": "Student_ID"}
)

academic_df = academic_df.rename(
    columns={"ID": "Student_ID"}
)

skills_df = skills_df.rename(
    columns={"student_id": "Student_ID"}
)

placement_df = placement_df.rename(
    columns={"StudentID": "Student_ID"}
)
```

Observation:

All datasets now use the common identifier name Student_ID.

Step 10: Standardize Identifier Data Type

The identifier was converted into a common numeric format.

Code used:

```
[13]
✓ Os
for df in [
    student_df,
    academic_df,
    skills_df,
    placement_df
]:
    df["Student_ID"] = pd.to_numeric(
        df["Student_ID"],
        errors="coerce"
    ).astype("Int64")
```

Observation:

The Student_ID field was standardized across all datasets, allowing reliable comparison and merging.

Step 11: Check Inconsistent Categorical Values

Categorical values were inspected for variations in spelling, capitalization, and whitespace.

Code used:

```
[14]
✓ Os
for name, df in {
    "Student Details": student_df,
    "Academic Performance": academic_df,
    "Technical Skills": skills_df,
    "Placement Details": placement_df
}.items():
    print(
        name,
        "→ Missing Student_ID:",
        df["Student_ID"].isna().sum()
    )
```

Student Details → Missing Student_ID: 0
 Academic Performance → Missing Student_ID: 5
 Technical Skills → Missing Student_ID: 5
 Placement Details → Missing Student_ID: 5

Observation:

Inconsistent categorical representations were found in the Technical Skills dataset, such as variations in capitalization and trailing spaces.

Step 12: Standardize Required Categorical Fields

The relevant categorical fields were standardized by removing unnecessary spaces and converting values to uppercase.

Code used:

```
[15]
✓ Os
for name, df in {
    "Student Details": student_df,
    "Academic Performance": academic_df,
    "Technical Skills": skills_df,
    "Placement Details": placement_df
}.items():

    print("\n" + "="*60)
    print(name)

    for column in df.select_dtypes(
        include="object"
    ).columns:

        print("\n", column)
        print(df[column].dropna().unique()[:20])
```

```
=====
--- Student Details

Name
['Rahul Iyer' 'Rohan Singh' 'Rahul Menon' 'Surya Patel' 'Vikram Singh'
 'Ananya Singh' 'Arjun Rao' 'Aadhya Singh' 'Meera Rao' 'Aadhya Nair'
 'Aarav Menon' 'Meera Das' 'Pranav Menon' 'Ishita Reddy' 'Meera Patel'
 'Divya Das' 'Arjun Nair' 'Kavin Patel' 'Keerthi Iyer' 'Nithya Sharma']

Gender
['Male' 'Female']

Department
['AI&DS' 'CIVIL' 'ECE' 'CSE' 'EEE' 'MECH']

=====
Academic Performance

=====
Technical Skills

Programming_Language
['Python' 'Python' 'Java' 'c' 'C++' 'JAVA']

SQL_Level
['Advanced' 'Intermediate' 'Beginner' 'advanced' 'intermediate']

Communication
['Excellent' 'Average' 'Good' 'good' 'excellent']

Certification
['Udemy' 'NPTEL' 'IBM' 'Coursera']

=====
Placement Details

Company
['TCS' 'Deloitte' 'Cognizant' 'Accenture' 'Zoho' 'Wipro' 'Infosys']

Placement_Status
['Interview' 'Not Placed' 'Placed']
```

```
[17]
✓ Os
skills_df["Programming_Language"] = (
    skills_df["Programming_Language"]
    .astype("string")
    .str.strip()
    .str.upper()
)

skills_df["SQL_Level"] = (
    skills_df["SQL_Level"]
    .astype("string")
    .str.strip()
    .str.upper()
)

skills_df["Communication"] = (
    skills_df["Communication"]
    .astype("string")
    .str.strip()
    .str.upper()
)
```

Observation:

Values such as Python and Python were standardized to the same representation.

Step 13: Identify Unmatched Student Records

Student IDs in the master Student Details dataset were compared with the other datasets.

Code used:

```
[18]
✓ Os
student_ids = set(
    student_df["Student_ID"].dropna()
)

academic_ids = set(
    academic_df["Student_ID"].dropna()
)

skills_ids = set(
    skills_df["Student_ID"].dropna()
)

placement_ids = set(
    placement_df["Student_ID"].dropna()
)
```

```
[19] ✓ On  print(  
    "Student IDs missing from Academic dataset:",  
    len(student_ids - academic_ids)  
)  
  
print(  
    "Student IDs missing from Skills dataset:",  
    len(student_ids - skills_ids)  
)  
  
print(  
    "Student IDs missing from Placement dataset:",  
    len(student_ids - placement_ids)  
)  
  
*** Student IDs missing from Academic dataset: 20  
Student IDs missing from Skills dataset: 35  
Student IDs missing from Placement dataset: 105
```

Observation:

Some registered students do not have corresponding records in one or more datasets. These unmatched records were retained rather than deleted.

6. SELECTION OF COMMON INTEGRATION KEY

Three possible fields were considered:

- Student_ID
- Name
- Department

The uniqueness of these fields was compared.

Code used:

```
[20]
✓ Os
key_analysis = pd.DataFrame({
    "Key": ["Student_ID", "Name", "Department"],
    "Uniqueness": [
        student_df["Student_ID"].nunique(),
        student_df["Name"].nunique(),
        student_df["Department"].nunique()
    ],
    "Missing_Values": [
        student_df["Student_ID"].isna().sum(),
        student_df["Name"].isna().sum(),
        student_df["Department"].isna().sum()
    ]
})

display(key_analysis)
```

	Key	Uniqueness	Missing_Values
0	Student_ID	1000	0
1	Name	198	0
2	Department	6	0

Next steps: [Generate code with key_analysis](#) [New interactive sheet](#)

Observation:

Student_ID provides the most appropriate student-level identifier. Name and Department are not sufficiently unique for reliable integration.

Why Student_ID Was Selected

Student_ID was selected as the common integration key because it is specifically intended to identify an individual student.

Using **Name** is inappropriate because:

- Different students may have the same name.
- Names may contain spelling differences.
- Names may contain extra spaces or formatting variations.
- Names are not guaranteed to be unique.

Using **Department** is inappropriate because:

- Many students belong to the same department.
- Department identifies a group rather than an individual.
- Department cannot uniquely match student records.

Therefore, Student_ID is the most reliable key for integrating the datasets.

7. MERGE STRATEGY

Student_Details was treated as the master dataset because it contains the registered student population.

A **LEFT JOIN** was selected because the integration should preserve student records even when corresponding academic, skills, or placement information is unavailable.

7.1 Merge Student Details with Academic Performance

Code used:

```
integrated_df = pd.merge(  
    student_df,  
    academic_df,  
    on="Student_ID",  
    how="left"  
)
```

```
print(  
    "After Academic merge:",  
    integrated_df.shape  
)
```

Validation

```
print(  
    "Unique students:",  
    integrated_df["Student_ID"].nunique()  
)
```

```
print(  
    "Missing Student_ID:",  
    integrated_df["Student_ID"].isna().sum()  
)
```

Observation:

The merge was validated before proceeding to the next dataset.

7.2 Merge Technical Skills

Code used:

```
[21] ✓ Os duplicate_names = (  
    student_df["Name"]  
    .value_counts()  
)  
  
print(  
    "Names appearing more than once:"  
)  
  
display(  
    duplicate_names[  
        duplicate_names > 1  
    ].head(20)  
)
```

```
*** Names appearing more than once:  
      count  
      Name  
Rahul Das    10  
Pranav Menon  10  
Aarav Singh   10  
Rohan Kumar   9  
Rahul Iyer     9  
Rahul Menon   9  
Aarav Nair     9  
Arjun Kumar    9  
Diya Reddy     9  
Aditya Nair    9  
Ananya Menon   9  
Nithya Iyer    9  
Riya Sharma    9  
Surya Patel    8  
Nithya Singh   8  
Rohan Nair     8  
Pranav Singh   8  
Rohan Singh    8  
Pranav Kumar   8  
Aarav Patel    8  
  
dtype: int64
```

```
[22]
✓ Os      print(
          student_df["Department"].value_counts()
        )

Department
CSE      234
AI&DS    222
ECE      165
MECH     129
EEE      128
CIVIL    122
Name: count, dtype: int64
```

Observation:

The integrated dataset was checked after adding technical skill information.

7.3 Merge Placement Details

Code used:

```
[23]
✓ Os      integrated_df = pd.merge(
          student_df,
          academic_df,
          on="Student_ID",
          how="left"
        )

        print(
          "After Academic merge:",
          integrated_df.shape
        )

After Academic merge: (1000, 9)
```

```
[24]
✓ Os print(
    "Unique students:",
    integrated_df["Student_ID"].nunique()
)

print(
    "Missing Student_ID:",
    integrated_df["Student_ID"].isna().sum()
)
```

*** Unique students: 1000
Missing Student_ID: 0

```
[25]
✓ Os integrated_df = pd.merge(
    integrated_df,
    skills_df,
    on="Student_ID",
    how="left"
)

print(
    "After Skills merge:",
    integrated_df.shape
)
```

After Skills merge: (1000, 13)

```
[26]
✓ Os print(
    "Unique Student IDs:",
    integrated_df["Student_ID"].nunique()
)

print(
    "Duplicate complete rows:",
    integrated_df.duplicated().sum()
)
```

Unique Student IDs: 1000
Duplicate complete rows: 0

```
[26]
✓ Os print(
    "Unique Student IDs:",
    integrated_df["Student_ID"].nunique()
)

print(
    "Duplicate complete rows:",
    integrated_df.duplicated().sum()
)
```

*** Unique Student IDs: 1000
Duplicate complete rows: 0

```
[27]
✓ Os print(
    "Unique Student IDs:",
    integrated_df["Student_ID"].nunique()
)

print(
    "Duplicate complete rows:",
    integrated_df.duplicated().sum()
)
```

Unique Student IDs: 1000
Duplicate complete rows: 0

Observation:

Placement information was successfully integrated using the standardized Student_ID field.

8. FINAL VALIDATION

Code used:

```
[28] print("="*70)
✓ Os print("FINAL INTEGRATED DATASET VALIDATION")
print("="*70)

print("Final rows:",
      integrated_df.shape[0])

print("Final columns:",
      integrated_df.shape[1])

print("Unique Student_ID:",
      integrated_df["Student_ID"].nunique())

print("Missing Student_ID:",
      integrated_df["Student_ID"].isna().sum())

print("Duplicate complete rows:",
      integrated_df.duplicated().sum())

*** =====
FINAL INTEGRATED DATASET VALIDATION
=====
Final rows: 1000
Final columns: 13
Unique Student_ID: 1000
Missing Student_ID: 0
Duplicate complete rows: 0
```

```
[29] original_student_count = (
✓ Os     student_df["Student_ID"].nunique()
)

final_student_count = (
     integrated_df["Student_ID"].nunique()
)

print(
    "Original unique students:",
    original_student_count
)

print(
    "Final unique students:",
    final_student_count
)

if original_student_count == final_student_count:
    print("All registered students are represented.")
else:
    print("Some registered students are missing.")

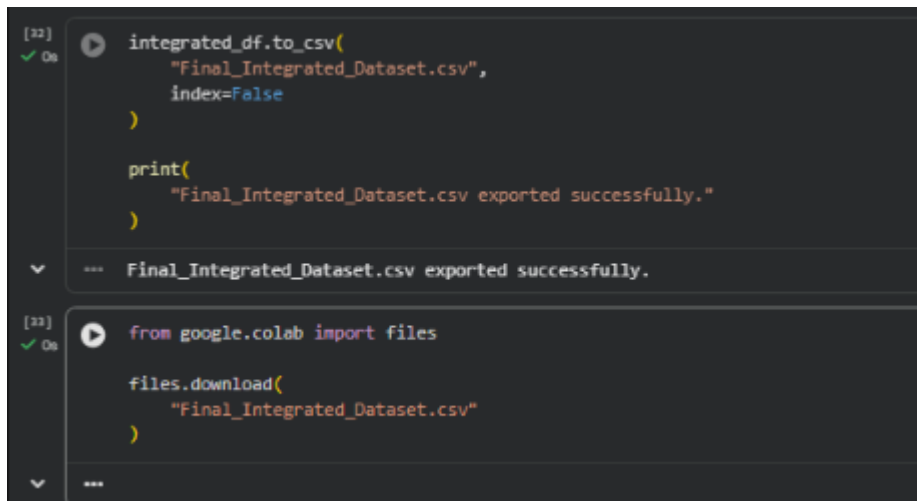
Original unique students: 1000
Final unique students: 1000
All registered students are represented.
```

Observation:

The final integrated dataset was validated to ensure that the student population from the master dataset remained represented after integration.

9. EXPORT FINAL DATASET

Code used:

A screenshot of a Google Colab notebook showing two code cells. The first cell, labeled [12], contains Python code to export a DataFrame to a CSV file. The code is:

```
integrated_df.to_csv("Final_Integrated_Dataset.csv", index=False)

print("Final_Integrated_Dataset.csv exported successfully.")
```

 The output of this cell is "Final_Integrated_Dataset.csv exported successfully." The second cell, labeled [13], contains code to download the file:

```
from google.colab import files

files.download("Final_Integrated_Dataset.csv")
```

 The output of this cell is empty.

Output:

Final_Integrated_Dataset.csv exported successfully.

10. RESULT

The four college student datasets were successfully inspected, cleaned, standardized, and integrated using Python and Pandas. Structural inconsistencies involving identifier names, identifier data types, missing values, duplicate identifiers, categorical values, and unmatched records were identified and handled without modifying the original CSV files.

Student_ID was selected as the most reliable common integration key because it provides student-level identification, whereas Name and Department are not guaranteed to uniquely identify individual students.

The integrated dataset was validated after each major merge and exported successfully as Final_Integrated_Dataset.csv.

11. CONCLUSION

The experiment demonstrated the importance of inspecting and standardizing datasets before integration. The analysis showed that selecting an appropriate common key is essential for accurate data integration. Student_ID was found to be more reliable than Name or Department because it is designed to uniquely represent an individual student. Pandas merge operations were then used to combine the datasets while preserving unmatched student records.

12. IMPORTANT OBSERVATIONS

1. Different datasets used different names for the same student identifier.
 2. Identifier data types were not consistent across all datasets.
 3. Missing and duplicate identifiers were detected.
 4. Some categorical values had capitalization and whitespace inconsistencies.
 5. Some student records were unmatched across datasets.
 6. Student_ID was the most suitable common integration key.
 7. A LEFT JOIN was appropriate because Student Details represented the master student population.
 8. The integrated dataset was validated after each major merge.
 9. Original CSV files were not manually modified.
 10. The final dataset was exported as Final_Integrated_Dataset.csv.
-

INVESTIGATION OF ONE-TO-MANY RELATIONSHIP IN STUDENT PLACEMENT DATA

Experiment Title

Analysis of Row Increase During Student and Placement Dataset Integration

1. AIM

To investigate the unexpected increase in records after integrating Student Details with Placement Details, identify duplicate Student_ID values, determine the relationship between the datasets, and establish whether the increased number of rows represents erroneous data or a legitimate one-to-many relationship.

2. OBJECTIVE

- Determine the number of records and unique students in the Student Details dataset.
 - Determine the number of records and unique students in the Placement Details dataset.
 - Identify students having multiple placement records.
 - Investigate the reason for the increase in rows after merging.
 - Demonstrate the one-to-many relationship between students and placement records.
 - Determine whether multiple placement records represent erroneous or legitimate data.
 - Validate the integrated dataset using row counts and unique Student_ID values.
-

3. INPUT DATASETS

Student_Details.csv

Contains information about 1,000 registered students.

Placement_Details_Q2_1275.csv

Contains 1,275 placement records associated with students.

The placement dataset intentionally represents a realistic placement scenario in which a student may have more than one placement record.

4. SOFTWARE USED

- Python 3.x
 - Google Colab
 - Pandas
 - NumPy
-

5. PROCEDURE

Step 1: Import Pandas

Pandas was imported to load, inspect, analyze, and merge the datasets.

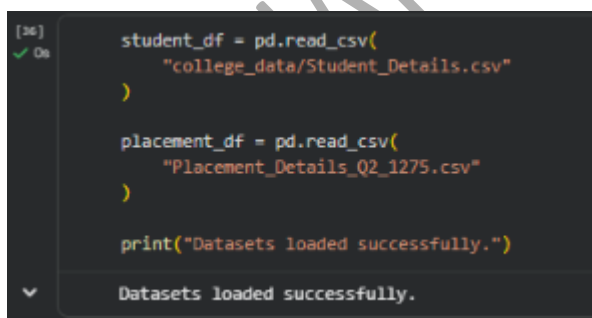
Code

```
import pandas as pd
```

Step 2: Load the Datasets

The Student Details and Placement Details datasets were loaded using Pandas.

Code



```
[36] In: student_df = pd.read_csv(
      "college_data/Student_Details.csv"
      )

      placement_df = pd.read_csv(
      "Placement_Details_Q2_1275.csv"
      )

      print("Datasets loaded successfully.")

Out: Datasets loaded successfully.
```

Step 3: Inspect Dataset Structure

The shape, columns, data types, and sample records were examined before performing the merge.

Code

```
[37] print("STUDENT DETAILS")
      print("="*60)
      print("Shape:", student_df.shape)
      print("Columns:", student_df.columns.tolist())
      print(student_df.dtypes)
      display(student_df.head())

      print("\nPLACEMENT DETAILS")
      print("="*60)
      print("Shape:", placement_df.shape)
      print("Columns:", placement_df.columns.tolist())
      print(placement_df.dtypes)
      display(placement_df.head())
```

Observation

The Student Details dataset contains 1,000 student records, while the Placement Details dataset contains 1,275 placement records.

STUDENT DETAILS					
Shape: (1000, 5)					
Columns: ['Student_ID', 'Name', 'Gender', 'Department', 'Year']					
Student_ID	int64				
Name	object				
Gender	object				
Department	object				
Year	int64				
dtype:	object				
Student_ID	Name	Gender	Department	Year	
0	1001	Rahul Iyer	Male	AI&DS	3
1	1002	Rohan Singh	Female	CIVIL	2
2	1003	Rahul Menon	Male	ECE	4
3	1004	Surya Patel	Male	CSE	4
4	1005	Vikram Singh	Female	EEE	3

PLACEMENT DETAILS					
Shape: (1275, 5)					
Columns: ['StudentID', 'Company', 'Package_LPA', 'Placement_Status', 'Placement_Year']					
StudentID	int64				
Company	object				
Package_LPA	float64				
Placement_Status	object				
Placement_Year	int64				
dtype:	object				
StudentID	Company	Package_LPA	Placement_Status	Placement_Year	
0	1132	Cognizant	5.1	Placed	2026
1	1204	Cognizant	8.0	Not Placed	2026
2	1051	Cognizant	4.4	Placed	2026
3	1586	TCS	9.5	Placed	2026
4	1139	Zoho	8.2	Interview	2026

Step 4: Standardize the Placement Identifier

The Student Details dataset uses Student_ID, while Placement Details uses StudentID. The identifier was renamed before integration.

Code

```
[18] placement_df = placement_df.rename(  
    columns={"StudentID": "Student_ID"  
})  
  
student_df["Student_ID"] = pd.to_numeric(  
    student_df["Student_ID"],  
    errors="coerce"  
).astype("Int64")  
  
placement_df["Student_ID"] = pd.to_numeric(  
    placement_df["Student_ID"],  
    errors="coerce"  
).astype("Int64")
```

Observation

Both datasets now use the common key Student_ID with a consistent data type.

6. INVESTIGATION OF RECORD COUNTS

Step 5: Compare Total Records and Unique Students

Code

```
[39] print("Student Details")  
    print("Total records:", len(student_df))  
    print("Unique Student_IDs:",  
          student_df["Student_ID"].nunique())  
  
    print("\nPlacement Details")  
    print("Total records:", len(placement_df))  
    print("Unique Student_IDs:",  
          placement_df["Student_ID"].nunique())  
  
    Student Details  
    Total records: 1000  
    Unique Student_IDs: 1000  
  
    Placement Details  
    Total records: 1275  
    Unique Student_IDs: 950
```

Observation

The Placement Details dataset contains 1,275 records but only 950 unique students. Therefore, some students have more than one placement record.

7. IDENTIFY DUPLICATE STUDENT IDs

Step 6: Count Placement Records Per Student

Code

```
[40] placement_counts = (
    placement_df
    .groupby("Student_ID")
    .size()
    .reset_index(name="Placement_Record_Count")
)

display(
    placement_counts
    .sort_values(
        "Placement_Record_Count",
        ascending=False
    )
    .head(20)
)
```

Student_ID	Placement_Record_Count
331	4
450	4
499	4
199	4
239	4
234	4
113	3
754	3
746	3
95	3
437	3
780	3
380	3
83	3
81	3
809	3
807	3
59	3
394	3
723	3

Step 7: Identify Students With Multiple Placement Records

Code

```
[41] multiple_placements = placement_counts[
    placement_counts["Placement_Record_Count"] > 1
]

print(
    "Students with multiple placement records:",
    len(multiple_placements)
)

*** Students with multiple placement records: 275
```

Observation

275 students have more than one placement record. This indicates that Student_ID is unique in the Student Details dataset but is not unique in the Placement Details dataset.

8. INSPECT ACTUAL MULTIPLE PLACEMENT RECORDS

Step 8: Display Students With Multiple Placement Records

Code

```
[42] ✓ On  duplicate_ids = multiple_placements["Student_ID"]

multiple_placement_records = placement_df[
    placement_df["Student_ID"].isin(duplicate_ids)
].sort_values("Student_ID")

display(
    multiple_placement_records.head(30)
)
```

	Student_ID	Company	Package_LPA	Placement_Status	Placement_Year
1168	1001	Wipro	4.2	Placed	2026
507	1001	Wipro	9.0	Placed	2026
665	1002	Accenture	11.5	Placed	2026
1080	1002	Deloitte	10.1	Placed	2026
887	1004	Accenture	4.8	Placed	2026
1009	1004	Infosys	6.6	Placed	2026
129	1005	Zoho	5.8	Placed	2026
1173	1005	Zoho	11.8	Placed	2026
825	1009	Zoho	4.2	Interview	2026
975	1009	Cognizant	11.0	Placed	2026
317	1010	Deloitte	10.9	Interview	2026
1117	1010	Zoho	12.9	Placed	2026
1243	1020	Accenture	9.3	Placed	2026
667	1020	Wipro	3.5	Not Placed	2026
660	1026	Wipro	7.4	Placed	2026
1165	1026	Accenture	9.9	Placed	2026
969	1034	Wipro	8.5	Placed	2026
998	1034	Cognizant	6.8	Placed	2026
469	1034	Cognizant	7.0	Interview	2026
167	1037	Wipro	6.5	Interview	2026
1075	1037	Deloitte	8.6	Placed	2026
1123	1041	Infosys	9.8	Placed	2026
347	1041	Infosys	8.6	Placed	2026
1181	1042	Deloitte	4.5	Placed	2026
93	1042	Infosys	9.6	Placed	2026
713	1045	Zoho	4.1	Placed	2026
994	1045	Cognizant	11.7	Placed	2026
1124	1047	Wipro	10.8	Placed	2026
860	1047	Accenture	10.9	Interview	2026
677	1054	Cognizant	7.3	Not Placed	2026

Observation

The output shows that the same student can appear in multiple placement records, potentially with different companies, packages, or placement events.

9. PERFORM THE MERGE

Step 9: Merge Student Details With Placement Details

A LEFT JOIN was used because Student Details represents the complete registered student population.

Code

```
[43] ✓ Os student_placement = pd.merge(
    student_df,
    placement_df,
    on="Student_ID",
    how="left"
)

print(
    "Merged dataset shape:",
    student_placement.shape
)

Merged dataset shape: (1325, 9)
```

10. VALIDATE THE MERGED DATASET

Step 10: Compare Rows and Unique Students

Code

```
[44] ✓ Os print("="*60)
print("FINAL VALIDATION")
print("="*60)

print("Original Student records:",
      len(student_df))

print("Merged records:",
      len(student_placement))

print("Original unique students:",
      student_df["Student_ID"].nunique())

print("Merged unique students:",
      student_placement["Student_ID"].nunique())

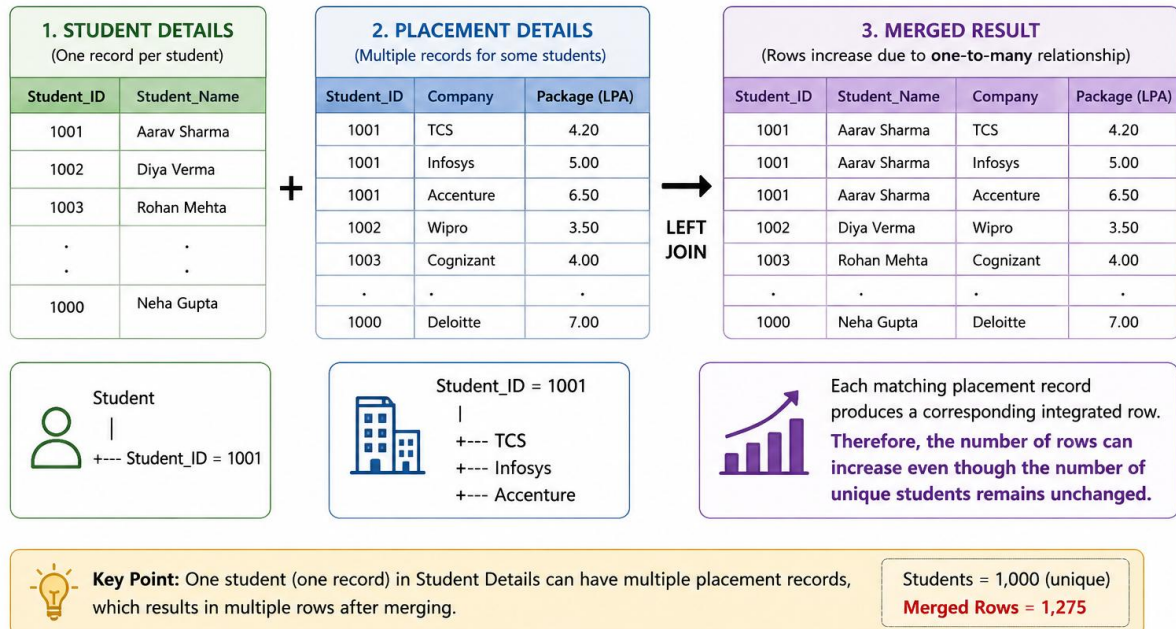
print("Students with multiple placement records:",
      len(multiple_placements))

***
=====
FINAL VALIDATION
=====
Original Student records: 1000
Merged records: 1325
Original unique students: 1000
Merged unique students: 1000
Students with multiple placement records: 275
```

11. EXPLAIN THE ROW INCREASE

EXPLAINING THE ROW INCREASE

The increase from **1,000 student records** to **1,275 merged records** occurs because the relationship between **Student Details** and **Placement Details** is **one-to-many**.



12. DETERMINE WHETHER THE INCREASE IS AN ERROR

The increase in records does not necessarily represent erroneous data.

The analysis shows that multiple placement records are associated with the same Student_ID. This is consistent with a legitimate one-to-many relationship in which a student can have multiple placement records, such as multiple company offers, interviews, or placement events.

Therefore, the additional rows should not be deleted merely because they increase the dataset size.

Deleting them could result in loss of valid placement information.

13 RESULT

The Student Details and Placement Details datasets were successfully integrated using Pandas.

The original Student Details dataset contained 1,000 records. After integration, the resulting dataset contained 1,275 records, while the number of unique Student_ID values remained 1,000.

Investigation showed that 275 students had multiple placement records. Therefore, the increase in rows is caused by a legitimate one-to-many relationship between students and placement records rather than simply by erroneous duplicate data.

14. CONCLUSION

The experiment demonstrated that an increase in row count after a merge does not necessarily indicate incorrect data. The relationship between Student Details and Placement Details was identified as one-to-many because a single student can have multiple placement records.

The evidence from the grouped Student_ID counts and the merged dataset confirmed that the 1,275 rows represent multiple placement records while the unique student population remains 1,000. Therefore, the additional records should be retained unless there is evidence that the placement records themselves are invalid.

INTEGRATION OF STUDENT, ACADEMIC, SKILLS AND PLACEMENT DATASETS

Experiment Title

Integration of Multiple Student Datasets Using an Appropriate Join Strategy

1. AIM

To integrate Student Details, Academic Performance, Technical Skills, and Placement Information while ensuring that all registered students remain in the final dataset, identify students whose academic information is unavailable, and justify the selection of the appropriate join strategy.

2. OBJECTIVES

- Inspect the number of records in each dataset.
- Compare the registered student population with the available academic records.
- Identify students whose academic information is unavailable.
- Standardize the common Student_ID field.

- Compare INNER JOIN and LEFT JOIN.
 - Select the join based on the stated data requirement.
 - Integrate all four datasets.
 - Validate that all registered students remain represented.
 - Export the final integrated dataset.
-

3. INPUT DATASETS

The following datasets were used:

1. Student_Details.csv
2. Academic_Performance_Q3_960.csv
3. Technical_Skills.csv
4. Placement_Details.csv

The Student Details dataset contains 1,000 registered students, while the Academic Performance dataset contains 960 records.

4. SOFTWARE USED

- Python 3.x
 - Google Colab
 - Pandas
 - NumPy
-

5. PROCEDURE

Step 1: Load the Datasets

All four CSV datasets were loaded using Pandas.

Output

```
[45] import pandas as pd
import numpy as np

[46] student_df = pd.read_csv(
    "college_data/Student_Details.csv"
)

academic_df = pd.read_csv(
    "Academic_Performance_Q3_960.csv"
)

skills_df = pd.read_csv(
    "college_data/Technical_Skills.csv"
)

placement_df = pd.read_csv(
    "college_data/Placement_Details.csv"
)

print("All four datasets loaded successfully.")

... All four datasets loaded successfully.
```

Step 2: Compare Dataset Sizes

The number of records in each dataset was examined.

Observation

The Student Details dataset contains 1,000 registered students, while the Academic Performance dataset contains 960 records. This indicates that academic information may be unavailable for some registered students.

```
[47] print("Student Details:", student_df.shape)
print("Academic Performance:", academic_df.shape)
print("Technical Skills:", skills_df.shape)
print("Placement Details:", placement_df.shape)

Student Details: (1000, 5)
Academic Performance: (960, 5)
Technical Skills: (970, 5)
Placement Details: (1020, 5)
```

Step 3: Standardize the Common Identifier

The Academic Performance dataset used ID, whereas the Student Details dataset used Student_ID. The academic identifier was renamed to Student_ID.

```
[33] ✓ Os # Standardize identifier column names

student_df = student_df.rename(columns={
    "ID": "Student_ID",
    "student_id": "Student_ID",
    "StudentID": "Student_ID"
})

academic_df = academic_df.rename(columns={
    "ID": "Student_ID",
    "student_id": "Student_ID",
    "StudentID": "Student_ID"
})

skills_df = skills_df.rename(columns={
    "ID": "Student_ID",
    "student_id": "Student_ID",
    "StudentID": "Student_ID"
})

placement_df = placement_df.rename(columns={
    "ID": "Student_ID",
    "student_id": "Student_ID",
    "StudentID": "Student_ID"
})

print("Identifier columns standardized successfully.")

print("\nStudent:", student_df.columns.tolist())
print("Academic:", academic_df.columns.tolist())
print("Skills:", skills_df.columns.tolist())
print("Placement:", placement_df.columns.tolist())

Identifier columns standardized successfully.

Student: ['Student_ID', 'Name', 'Gender', 'Department', 'Year']
Academic: ['Student_ID', 'CGPA', 'Attendance', 'Backlogs', 'Semester']
Skills: ['Student_ID', 'Programming_Language', 'SQL_Level', 'Communication', 'Certification']
Placement: ['Student_ID', 'Company', 'Package_LPA', 'Placement_Status', 'Placement_Year']
```

```
[54] ✓ Os

for df in [
    student_df,
    academic_df,
    skills_df,
    placement_df
]:
    df["Student_ID"] = pd.to_numeric(
        df["Student_ID"],
        errors="coerce"
    ).astype("Int64")

    print("Student_ID data types standardized successfully.")

Student_ID data types standardized successfully.
```

The identifier was also converted to a common numeric data type to ensure reliable matching.

Step 4: Identify Students Without Academic Records

The set of registered Student_ID values was compared with the Student_ID values available in the Academic Performance dataset.

Observation

The analysis identified **40 registered students whose academic information was unavailable**.

Step 5: Display the Missing Students

The 40 students were retrieved from the Student Details dataset using their Student_ID values.

Observation

The resulting table provides the Student_ID, name, and department of the 40 registered students for whom no academic record was available.

Step 5: Display the Missing Students

The 40 students were retrieved from the Student Details dataset using their Student_ID values.

Observation

The resulting table provides the Student_ID, name, and department of the 40 registered students for whom no academic record was available.

```
[sa]
✓ Os
students_without_academics = student_df[
    student_df["Student_ID"].isin(
        missing_academic_ids
    )
]

display(
    students_without_academics
)
```

	Student_ID	Name	Gender	Department	Year
17	1018	Surya Patel	Female	AI&DS	4
25	1026	Rohan Reddy	Male	CIVIL	4
27	1028	Ananya Sharma	Female	EEE	4
28	1029	Riya Iyer	Male	CSE	2
47	1048	Harini Patel	Female	ECE	4
77	1078	Aarav Reddy	Female	AI&DS	4
116	1117	Ishita Nair	Male	AI&DS	2
135	1136	Ishita Iyer	Female	EEE	3
201	1202	Nithya Iyer	Female	MECH	3
257	1258	Ishita Das	Female	CIVIL	4
294	1295	Keerthi Das	Female	CSE	3
324	1325	Aditya Nair	Female	ECE	2
333	1334	Riya Patel	Female	CIVIL	4
361	1362	Ananya Singh	Female	CIVIL	3
390	1391	Rahul Das	Male	AI&DS	3
412	1413	Kavin Patel	Male	CIVIL	2
413	1414	Arjun Reddy	Male	AI&DS	4
417	1418	Divya Iyer	Male	CSE	2
435	1436	Sanjay Singh	Female	EEE	3
438	1439	Kavin Kumar	Male	CSE	3
463	1464	Surya Patel	Female	CSE	2
480	1481	Aditya Iyer	Male	EEE	3
541	1542	Surya Sharma	Female	AI&DS	3

6. JOIN STRATEGY SELECTION

The requirement states:

Every registered student must remain in the final dataset.

Therefore, Student Details is treated as the master dataset.

Two possible join strategies were considered.

```
[59]
✓ Os      check = academic_df[
          academic_df["Student_ID"].isin(
              missing_academic_ids
          )
          ]

          print(
              "Academic records found for missing students:",
              len(check)
          )

          Academic records found for missing students: 0

[60]
✓ Os      print("Requirement:")
          print(
              "Every registered student must remain "
              "in the final dataset."
          )

          Requirement:
          Every registered student must remain in the final dataset.
```

INNER JOIN

An INNER JOIN retains only records that have matching Student_ID values in both datasets.

In this case, the Academic Performance dataset contains 960 students. Therefore, an INNER JOIN would retain approximately 960 students and remove the 40 students without academic records.

Result

INNER JOIN unique students = 960

This violates the requirement that all 1,000 registered students must remain.

```
[61]
✓ Os      inner_result = pd.merge(
          student_df,
          academic_df,
          on="Student_ID",
          how="inner"
          )

          print(
              "INNER JOIN rows:",
              len(inner_result)
          )

          print(
              "INNER JOIN unique students:",
              inner_result["Student_ID"].nunique()
          )

          INNER JOIN rows: 960
          INNER JOIN unique students: 960
```

LEFT JOIN

A LEFT JOIN retains every record from the left dataset, which is Student Details, and attaches matching academic information when available.

Therefore:

Student Details = 1,000 students

LEFT JOIN Academic Performance

↓

Final student population = 1,000 students

The 40 students without academic information remain in the dataset, with their academic fields represented as missing values (NaN).

```
[62]
✓ Os
student_academic = pd.merge(
    student_df,
    academic_df,
    on="Student_ID",
    how="left"
)

print(
    "LEFT JOIN rows:",
    len(student_academic)
)

print(
    "LEFT JOIN unique students:",
    student_academic["Student_ID"].nunique()
)

LEFT JOIN rows: 1000
LEFT JOIN unique students: 1000
```

7. JUSTIFICATION FOR LEFT JOIN

A LEFT JOIN was selected because the requirement is to retain **every registered student**.

The join was not selected simply because it produced more records. It was selected because it satisfies the actual data-integration requirement.

The 40 students without academic records should not be deleted because the absence of academic information is itself meaningful information.

Therefore:

Student Details + Academic Performance → LEFT JOIN

is the appropriate strategy.

```
[64] student_academic_skills = pd.merge(  
    student_academic,  
    skills_df,  
    on="Student_ID",  
    how="left"  
)  
  
print(  
    "After Skills merge:",  
    student_academic_skills.shape  
)  
  
print(  
    "Unique students:",  
    student_academic_skills["Student_ID"].nunique()  
)  
  
--- After Skills merge: (1000, 13)  
Unique students: 1000  
  
[65] final_df = pd.merge(  
    student_academic_skills,  
    placement_df,  
    on="Student_ID",  
    how="left"  
)  
  
print(  
    "After Placement merge:",  
    final_df.shape  
)  
  
print(  
    "Unique students:",  
    final_df["Student_ID"].nunique()  
)  
  
--- After Placement merge: (1120, 17)  
Unique students: 1000
```

8. INTEGRATION WITH SKILLS AND PLACEMENT DATA

After integrating Student Details and Academic Performance using a LEFT JOIN, Technical Skills and Placement Information were also integrated using Student_ID.

LEFT JOINS were used with Student Details as the master population so that students were not removed merely because information was unavailable in another dataset.

The Placement dataset may contain multiple records for some students. Therefore, the final row count may exceed 1,000 even though the number of unique students remains 1,000.

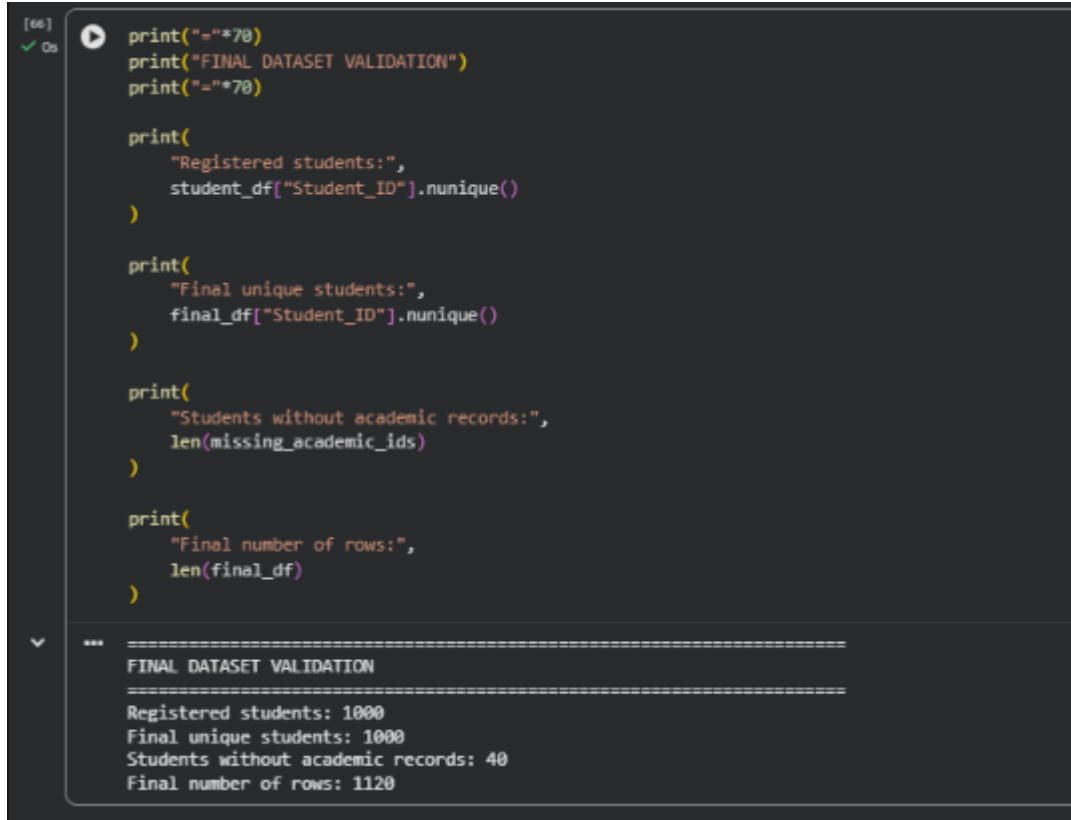
9. FINAL VALIDATION

The final dataset was validated by comparing the original number of registered students with the number of unique Student_ID values in the integrated dataset.

Observation

The validation confirmed that all **1,000 registered students** remain represented in the final integrated dataset.

The analysis also confirmed that **40 students do not have corresponding academic records**.



```
[66] In
✓ Out
print("="*70)
print("FINAL DATASET VALIDATION")
print("="*70)

print(
    "Registered students:",
    student_df["Student_ID"].nunique()
)

print(
    "Final unique students:",
    final_df["Student_ID"].nunique()
)

print(
    "Students without academic records:",
    len(missing_academic_ids)
)

print(
    "Final number of rows:",
    len(final_df)
)

...
=====
FINAL DATASET VALIDATION
=====
Registered students: 1000
Final unique students: 1000
Students without academic records: 40
Final number of rows: 1120
```

10. RESULT

The four datasets were successfully integrated using Pandas. The 40 registered students whose academic information was unavailable were identified by comparing Student_ID values.

A LEFT JOIN was selected because the requirement was to retain all 1,000 registered students. Unlike an INNER JOIN, the LEFT JOIN preserved the 40 students without academic records while representing their unavailable academic information as missing values.

11. CONCLUSION

The experiment demonstrated that join selection must be based on the data requirement rather than simply on the number of records produced. Since every registered student was required to remain in the final dataset, a LEFT JOIN was the appropriate strategy.

The analysis identified 40 students without academic records and successfully retained them in the integrated dataset. The final dataset therefore preserves the complete registered student population while clearly representing unavailable academic information.

JEYA HARSHINI R 192424051